Lazy Propagation

Range updates and range queries in O(log n): postpone work until someone looks

Learning goals

- See why point-update segment trees fail for **range updates**
- Understand **lazy tags**: "this whole subtree still owes an update"
- Implement **push** (propagate) and **pull** (recompute)
- Handle **range add + range sum/min**, **range assign**, and **assign + add** together
- Reason about **tag composition** and the order of operations
- Recognize lazy problems on flattened trees (Week 22)

Motivating problem — CSES 1735

Range Updates and Sums — n, q ≤ 2·10^5

- `1 a b x` — add x to every element in `[a, b]`
- `2 a b x` — set every element in `[a, b]` to x
- `3 a b` — print the sum of `[a, b]`

Point updates one by one: O(n log n) per query → 4·10^10. Too slow.

The lazy idea

A range update covers O(log n) canonical nodes (same as a query).

- At each **covered** node: fix its stored value **now**, and leave a note (**tag**) "my children still need this update"
- Do **not** descend further
- Children get the update **only when someone visits them** later

Like a manager who updates a team total and tells the team details later.

Picture: add 5 on [2, 7] (n = 8)



Only 2 covered nodes + their ancestors are touched. Ancestors are recomputed on the way back up.

Two helper operations

apply(x, v) — apply update `v` to node `x` as a whole:

- fix `sum[x]` (needs the segment length!)
- merge `v` into `tag[x]`

push(x) — before descending into `x`'s children:

- `apply(left, tag[x])`, `apply(right, tag[x])`
- clear `tag[x]`

pull(x) — after children change: `sum[x] = sum[2x] + sum[2x+1]`

Range add + range sum: apply & push

```
void apply(int x, int l, int r, long long v) {
    sum[x] += v * (r - l + 1);
    lz[x] += v;
}
void push(int x, int l, int r) {
    if (lz[x] != 0) {
        int m = (l + r) / 2;
        apply(2*x, l, m, lz[x]);
        apply(2*x+1, m+1, r, lz[x]);
        lz[x] = 0;
    }
}
```

Update and query

```
void update(int x, int l, int r, int ql, int qr, long long v) {
    if (qr < l || r < ql) return;
    if (ql <= l && r <= qr) { apply(x, l, r, v); return; }
    push(x, l, r);
    int m = (l + r) / 2;
    update(2*x, l, m, ql, qr, v);
    update(2*x+1, m+1, r, ql, qr, v);
    sum[x] = sum[2*x] + sum[2*x+1];          // pull
}
long long query(int x, int l, int r, int ql, int qr) {
    if (qr < l || r < ql) return 0;
    if (ql <= l && r <= qr) return sum[x];
    push(x, l, r);
    int m = (l + r) / 2;
    return query(2*x, l, m, ql, qr) + query(2*x+1, m+1, r, ql, qr);
}
```

Invariant

> `sum[x]` is always **correct** for its segment, taking into account all updates applied to `x` or above it **that have reached x**.
> `tag[x]` = updates that reached `x` but **not yet** its children.

- A node's value never depends on tags of its **descendants**
- So `push` before reading children, `pull` after changing them
- Complexity: same 4 nodes per level as a query → **O(log n)**

Other combinations: range add + range min

```
void apply(int x, long long v) { mn[x] += v; lz[x] += v; }
// pull: mn[x] = min(mn[2x], mn[2x+1])
```

- Adding v to every element adds v to the minimum
- No length needed!
- CF 52C *Circular RMQ*

Range assign + range sum

Tag = "set everything to `v`" (or **none**):

```
void apply(int x, int len, long long v) {
    sum[x] = v * len;
    hasSet[x] = true; setVal[x] = v;      // overwrites older tag
}
```

- Need a **"no tag"** marker (a flag, or a sentinel value that cannot appear)
- New assignment **replaces** the old tag

The hard part: composing tags

A tag must summarize **any sequence** of pending updates in O(1) space.

Pending sequence	Composed tag
add a, then add b	add a+b
set s, then add a	set s, then add a
add a, then set s	set s (add is wiped)
set s, then set t	set t

Normal form: **(optional set s) then (add a)** — closed under composition ✔

Assign + add: apply

```
struct Tag { bool hasSet = false; long long setVal = 0, add = 0; };

void applyTag(int x, int len, const Tag& t) {
    if (t.hasSet) {                       // assignment wipes everything below
        sum[x] = t.setVal * len;
        tag[x] = {true, t.setVal, 0};
    }
    if (t.add != 0) {                     // add stacks on top
        sum[x] += t.add * len;
        tag[x].add += t.add;
    }
}
```

Order inside `applyTag` mirrors the normal form: set first, then add.

Trace on CSES 1735 sample

```
a = 2 3 1 1 5 3
3 3 5     → 1+1+5 = 7
1 2 4 2   → a = 2 5 3 3 5 3
3 3 5     → 3+3+5 = 11
2 2 4 5   → a = 2 5 5 5 5 3
3 3 5     → 5+5+5 = 15
```

Output `7 11 15` ✔ — but internally most of those changes are still sitting in tags.

General recipe (monoid + action)

Ask three questions:

1. **Node value**: what is combined? (`sum`, `min`, `(sum, len)`, ...)
2. **Tag**: how does one update act on a node's value? `apply(value, tag)`
3. **Composition**: `tag_new ∘ tag_old` must be a tag of the same shape

Examples of tags closed under composition: add, assign, affine `x → bx + c` (sum mod p), xor-with-mask (per bit counts), flip.

Affine updates

Range `x → b·x + c`, range sum mod p:

- apply: `sum = b·sum + c·len`
- compose (old `(b1,c1)`, then new `(b2,c2)`): `x → b2(b1x + c1) + c2` = `(b2·b1, b2·c1 + c2)`
- Covers add (`b = 1`), multiply (`c = 0`), assign (`b = 0`)

Library Checker *Range Affine Range Sum* is the standard test.

When the node needs more

- **Arithmetic progression add** (CSES 1736 *Polynomial Queries*): tag = (first term, difference); sum of AP over the segment
- **Bit flip + count ones** (CF 242E *XOR on Segment*): keep counts per bit, xor tag flips counts `cnt → len − cnt`
- **Sort substring** (CF 558E): 26 assign-trees over counts

If you cannot define a compact tag, consider other techniques (sqrt decomposition, Week 34).

Lazy on trees

Euler tour (Week 22) turns subtrees into ranges:

- "Add x to every node in the subtree of v" → range add on `[tin, tout]`
- "Recolor subtree, count distinct colors" (CF 620E *New Year Tree*): assign tag, node = **bitmask** of colors (≤ 60), combine = OR
- CF 343D *Water Tree*: assign on subtree + point updates

Alternatives without a lazy tree

Operations	Simpler tool
range add, point query	BIT on differences (Week 24)
range add, range sum	two BITs (Week 24)
range add, range min/max	lazy segment tree
range assign	lazy tree, or `std::map` of intervals
range update, offline	difference array (Week 7)

Complexity

	Time	Memory
build	O(n)	4n values + 4n tags
range update	O(log n)	
range query	O(log n)	

n = q = 2·10^5: ~10^7 node visits — fine. Use `ios::sync_with_stdio(false)`.

Common pitfalls

- Forgetting `push` in **query** (not only in update)
- Forgetting `pull` after recursive update
- `apply` that forgets to **multiply by segment length**
- Assignment tag "none" encoded as 0, but 0 is a valid value
- Wrong composition order (add then set ≠ set then add)
- Pushing at a **leaf** (children do not exist → out of bounds with 2n arrays)
- Overflow: `v * len` with `v` up to 10^9 and len 2·10^5 → `long long`

Practice problems

- CSES 1651 *Range Update Queries* (warm-up; BIT works too)
- CSES 1735 *Range Updates and Sums* (assign + add)
- CSES 1736 *Polynomial Queries* (arithmetic progressions)
- Codeforces 52C *Circular RMQ*
- Codeforces 242E *XOR on Segment*
- Codeforces 145E *Lucky Queries*
- Codeforces 620E *New Year Tree*
- Codeforces 558E *A Simple Task*
- Codeforces EDU Segment Tree part 2 — https://codeforces.com/edu/course/2/lesson/5

Summary

- Lazy propagation = **apply now at covered nodes, push later on demand**
- Three functions: `apply`, `push`, `pull`
- Design: node value + tag + **tag composition**
- Assign + add: normal form "(set) then (add)"
- Still O(log n) per operation
- Next week: back to DP — bitmask DP